

RELATÓRIO TÉCNICO CONFIDENCIAL

Análise de Produção Copa Fácil

Arquitetura · Segurança · Readiness para Deploy

Data: 28 de julho de 2026

Versão: 1.0

Branch: main · bf3e1da

1. Visão Geral da Arquitetura

MONOREPO pnpm workspaces 3 apps: backend · frontend · mobile	BACKEND NestJS 10 · Drizzle ORM · PostgreSQL 16 Redis · BullMQ · MinIO · Socket.io
FRONTEND Next.js 15 (App Router) · Zustand Axios · Tailwind CSS · shadcn/ui	INFRAESTRUTURA Docker multi-stage · Nginx (TLS/HTTP2) Prometheus · Grafana · GitHub Actions CI/CD
MULTI-TENANCY Schema-per-tenant no PostgreSQL AsyncLocalStorage para propagação do contexto	MOBILE Expo + React Native ⚠️ Apenas esqueleto — sem telas implementadas

A arquitetura é bem estruturada, com separação clara entre camadas (DDD/Clean Architecture), pipeline de segurança em múltiplas camadas e isolamento real de dados por tenant via `SET LOCAL search_path` dentro de transações PostgreSQL.

2. Resumo Executivo de Vulnerabilidades

1

CRÍTICA

3

ALTA

4

MÉDIA

3

BAIXA

3. Vulnerabilidades de Segurança

Crítica

VUL-01 · Segredos commitados no repositório git

Os arquivos abaixo contêm segredos reais (ou de desenvolvimento) e provavelmente estão no histórico do git:

Arquivo	Segredos expostos
<code>.env</code> (raiz)	DATABASE_URL com senha
<code>apps/backend/.env</code>	JWT_SECRET, JWT_REFRESH_SECRET, MINIO_ACCESS_KEY, MINIO_SECRET_KEY, MP_WEBHOOK_SECRET
<code>apps/frontend/.env.local</code>	JWT_SECRET, PORTAL_API_KEY_DEMO (key real exposta)
<code>docker-compose.yml</code>	JWT_SECRET em texto claro no YAML

Mesmo sendo segredos de desenvolvimento, ficam acessíveis a qualquer pessoa com acesso ao histórico do repositório. Uma vez que uma key vaza, ela deve ser tratada como comprometida.

COMO CORRIGIR

- Remover do histórico completo com `git filter-repo --path apps/backend/.env --invert-paths` (ou BFG Repo Cleaner) para cada arquivo comprometido.
- Revogar e rotacionar *todas* as keys listadas acima.
- Criar `.env.production` fora do repositório com segredos fortes e referenciá-lo apenas via `env_file` no compose de produção.
- Adicionar ao `.gitignore`: `*.env`, `.env.*`, `.env.local`, `!.env.example`.

Altas

VUL-02 · Upload valida apenas o Content-Type declarado (não o conteúdo real do arquivo)

O `UploadController` valida o `mimetype` que vem do header HTTP do cliente — e não do conteúdo binário do arquivo. Um atacante pode enviar um arquivo executável ou PHP com `Content-Type: image/jpeg` e ele passa pela validação.

COMO CORRIGIR

Instalar `file-type` e verificar os magic bytes do buffer antes de aceitar:

```
import { fileTypeFromBuffer } from 'file-type';

const detected = await fileTypeFromBuffer(file.buffer);
if (!detected || !ALLOWED_MIME_TYPES.includes(detected.mime)) {
  throw new BadRequestException('Tipo de arquivo inválido');
}
```

VUL-03 · Dados de CPF/documento armazenados em texto plano (não-conformidade LGPD)

O campo `players.document` (CPF/passaporte) é armazenado sem criptografia no banco de dados do tenant. CPF é dado pessoal sensível coberto pela LGPD. O projeto já possui um `EncryptionService` com AES-256-GCM implementado — ele não está sendo aplicado aqui.

COMO CORRIGIR

Usar `EncryptionService.encrypt()` ao salvar o campo e `.decrypt()` ao lê-lo. Criar uma migration para criptografar dados já existentes.

VUL-04 · Discrepância de validação de senha entre frontend e backend

O schema Zod do frontend aceita senhas com mínimo de **6 caracteres**, enquanto o backend (`Password.createRaw()`) exige **8 caracteres** com maiúscula, minúscula, número e caractere especial. O formulário aceita a senha, o backend rejeita — o usuário não recebe feedback claro sobre o motivo.

COMO CORRIGIR

Atualizar o `loginSchema` e o `registerSchema` Zod no frontend para `.min(8)` com os mesmos requisitos de complexidade aplicados no backend.

Médias

VUL-05 · Webhook de pagamento alternativo silencia erros de assinatura com HTTP 200

O `PaymentWebhooksController` (`/webhooks/payments/:provider`) retorna HTTP 200 ao receber uma assinatura inválida, para evitar infinite retry dos provedores. Isso torna impossível detectar ataques de webhook forjados nos logs ou métricas.

COMO CORRIGIR

Manter o HTTP 200 na resposta, mas registrar como `logger.warn()` com contexto (IP, provider, hash recebido vs esperado, timestamp). Adicionar métrica de contador `webhook.signature_invalid` no Prometheus.

VUL-06 · CSP com `unsafe-inline` para scripts

O Nginx envia `Content-Security-Policy: script-src 'self' 'unsafe-inline'`. Isso enfraquece significativamente a proteção contra XSS injetado, pois qualquer script inline é executado sem validação.

COMO CORRIGIR

Implementar nonces via `middleware.ts` do Next.js e repassá-los via header `x-nonce`. Configurar a CSP para usar `'nonce-{valor}'` em vez de `'unsafe-inline'`.

VUL-07 · Cache do TenantRegistry em memória — incompatível com múltiplas instâncias

O `TenantRegistryService.provisionedSchemas` é um `Set<string>` em memória. Em ambientes com múltiplas instâncias (horizontal scaling), cada pod tem seu próprio cache independente. Não é um bug de segurança (as queries usam `IF NOT EXISTS`), mas causa overhead e pode gerar race conditions.

COMO CORRIGIR

Mover o cache para Redis com TTL configurável (ex: 5 minutos): `redis.set('tenant:provisioned:{schema}', '1', 'EX', 300)`.

VUL-08 · Swagger pode ser exposto em ambientes de staging

A documentação Swagger é desabilitada apenas quando `NODE_ENV === 'production'`. Se um ambiente de staging usar `NODE_ENV=development`, todos os endpoints, schemas e exemplos ficam publicamente acessíveis.

COMO CORRIGIR

Adicionar uma variável de ambiente independente: `SWAGGER_ENABLED=false`. A inicialização do Swagger deve checar essa flag em vez (ou além) do `NODE_ENV`.

VUL-09 · CORS_ORIGINS inclui porta do Grafana desnecessariamente

A lista `CORS_ORIGINS` no `apps/backend/.env` inclui `http://localhost:3002` (Grafana). O Grafana não realiza chamadas XHR para a API — essa origem é ruído e pode gerar confusão em auditorias.

COMO CORRIGIR

Remover `http://localhost:3002` da lista de origens permitidas.

VUL-10 · Refresh token retornado no body da resposta

O `access_token` é armazenado em cookie `httpOnly` (seguro), mas o `refreshToken` é retornado no body da resposta e precisa ser gerenciado pelo cliente. Embora o Zustand não o persista em `localStorage`, ele permanece em memória JS — vulnerável a XSS no mesmo processo de renderização.

COMO CORRIGIR

Definir o refresh token também como cookie `httpOnly` separado (`Set-Cookie: refresh_token=...; HttpOnly; Secure; SameSite=Strict; Path=/auth/refresh`). O `Path` restrito garante que só seja enviado ao endpoint de refresh.

VUL-11 · App mobile sem implementação

O app mobile (`apps/mobile`) contém apenas um store de autenticação e definições de types. Não há telas, navegação ou chamadas à API implementadas. Se o mobile estiver no escopo do lançamento, é um bloqueador.

4. Checklist de Produção

Obrigatório (bloqueante para o deploy)

#	Item	Referência	Status
1	Remover segredos do histórico git (<code>git filter-repo</code> ou BFG)	VUL-01	Pendente
2	Revogar e rotacionar todas as keys/segredos expostos	VUL-01	Pendente
3	Criar <code>.env.production</code> com segredos fortes fora do repositório	VUL-01	Pendente
4	Adicionar <code>.env*</code> e <code>.env.local</code> ao <code>.gitignore</code>	VUL-01	Pendente
5	Validar magic bytes no upload de arquivos (<code>file-type</code>)	VUL-02	Pendente
6	Criptografar campo <code>players.document</code> com <code>EncryptionService</code>	VUL-03	Pendente
7	Alinhar validação de senha no frontend para <code>min(8)</code>	VUL-04	Pendente
8	Configurar domínio real e certificado TLS no <code>nginx.conf</code>	—	Pendente
9	Testar fluxo completo de deploy via <code>cd.yml</code> + health check	—	Pendente
10	Definir <code>ENCRYPTION_KEY</code> (64 hex chars) para dados sensíveis	—	Pendente

Importante (antes do primeiro usuário real)

#	Item	Referência	Status
11	Configurar SMTP real para e-mails transacionais (não Ethereum)	—	Pendente
12	Configurar <code>MP_ACCESS_TOKEN</code> de produção do Mercado Pago	—	Pendente
13	Testar fluxo completo de pagamento em sandbox antes de prod	—	Pendente
14	Configurar <code>BACKUP_ENCRYPTION_PASSWORD</code> para backups PostgreSQL	—	Pendente
15	Implementar renovação automática do certificado TLS (cron + certbot)	—	Pendente
16	Adicionar log de <code>warn</code> + métrica Prometheus em assinatura inválida de webhook	VUL-05	Pendente
17	Adicionar variável <code>SWAGGER_ENABLED</code> independente do <code>NODE_ENV</code>	VUL-08	Pendente

Recomendado (pode ser pós-lançamento)

#	Item	Referência
18	Implementar nonces na CSP para remover <code>unsafe-inline</code>	VUL-06
19	Definir refresh token em cookie httpOnly com <code>Path=/auth/refresh</code>	VUL-10
20	Migrar cache do TenantRegistry para Redis	VUL-07
21	Remover <code>localhost:3002</code> (Grafana) da lista CORS	VUL-09
22	Implementar app mobile (sem telas atualmente)	VUL-11
23	Política de privacidade + DPO para conformidade LGPD	—
24	Testes E2E do fluxo crítico (pagamento + campeonato)	—

5. Ordem de Execução Recomendada

Execute os itens na ordem abaixo para eliminar riscos de segurança antes de qualquer exposição do ambiente de produção:

- 1 **git filter-repo** — remover histórico de arquivos com segredos; revogar e rotacionar todas as keys expostas
- 2 Criar `.env.production` fora do repositório com segredos fortes (JWT mínimo 64 chars, ENCRYPTION_KEY 64 hex chars, MinIO keys únicas)
- 3 Atualizar `.gitignore` para bloquear qualquer arquivo `.env*` exceto `.env.example`
- 4 Magic bytes no upload — instalar `file-type` e validar o buffer antes de aceitar qualquer arquivo
- 5 Criptografar `players.document` usando o `EncryptionService` já existente + migration para dados existentes
- 6 Alinhar validação de senha — atualizar schemas Zod do frontend para `min(8)` com complexidade
- 7 Configurar servidor e TLS — domínio real, certificado Let's Encrypt, certbot automático
- 8 Testar deploy completo — `cd.yml` + health check + rollback automático em ambiente de staging antes de produção

6. O que Está Bem Implementado

Estes pontos representam práticas de segurança corretas que não devem ser revertidas ou simplificadas:

- ✓ **Multi-tenancy real** — `SET LOCAL search_path` dentro de transação + `AsyncLocalStorage` ; isolamento genuíno entre tenants sem vazamento de dados
- ✓ **JWT com blacklist Redis por jti** + token rotation no refresh — revogação real de tokens individuais, não apenas expiração
- ✓ **Rate limiting em 3 camadas** — Nginx (30 req/s) → ThrottlerModule NestJS → Redis Lua script por API key; dificulta ataques de força bruta e DDoS
- ✓ **Timing-safe em todas as operações críticas** — `timingSafeEqual` em webhooks MP, `bcrypt.compare()` no login, tempo constante no reset de senha
- ✓ **Prevenção de enumeração de usuário** — `bcrypt.hash(token)` executado mesmo quando o usuário não existe, equalizando o tempo de resposta
- ✓ **Cookies httpOnly/Secure/SameSite=Strict** em produção para o `access_token` ; tokens não acessíveis por JavaScript
- ✓ **ValidationPipe com whitelist + forbidNonWhitelisted** — campos não declarados nos DTOs são removidos ou rejeitados; `disableErrorMessage` em produção
- ✓ **Usuários não-root nos Dockerfiles** + imagens multi-stage mínimas — princípio de menor privilégio nos containers
- ✓ **CI com pnpm audit --audit-level=high** antes de qualquer deploy — vulnerabilidades em dependências bloqueiam o pipeline
- ✓ **Rollback automático no cd.yml** se o health check falhar após o deploy — proteção contra regressões em produção

✓ **Hash SHA-256 das API keys no banco** (não a key em si) + cache Redis de 5 min — API keys não comprometidas mesmo com dump do banco

✓ **Validação de variáveis de ambiente via Zod** no boot — a aplicação não sobe silenciosamente com configuração inválida ou incompleta